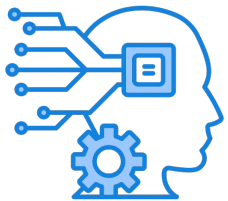
Universidade Federal de Goiás
Instituto de Informática
Prof. Dr. Ronaldo Martins da Costa

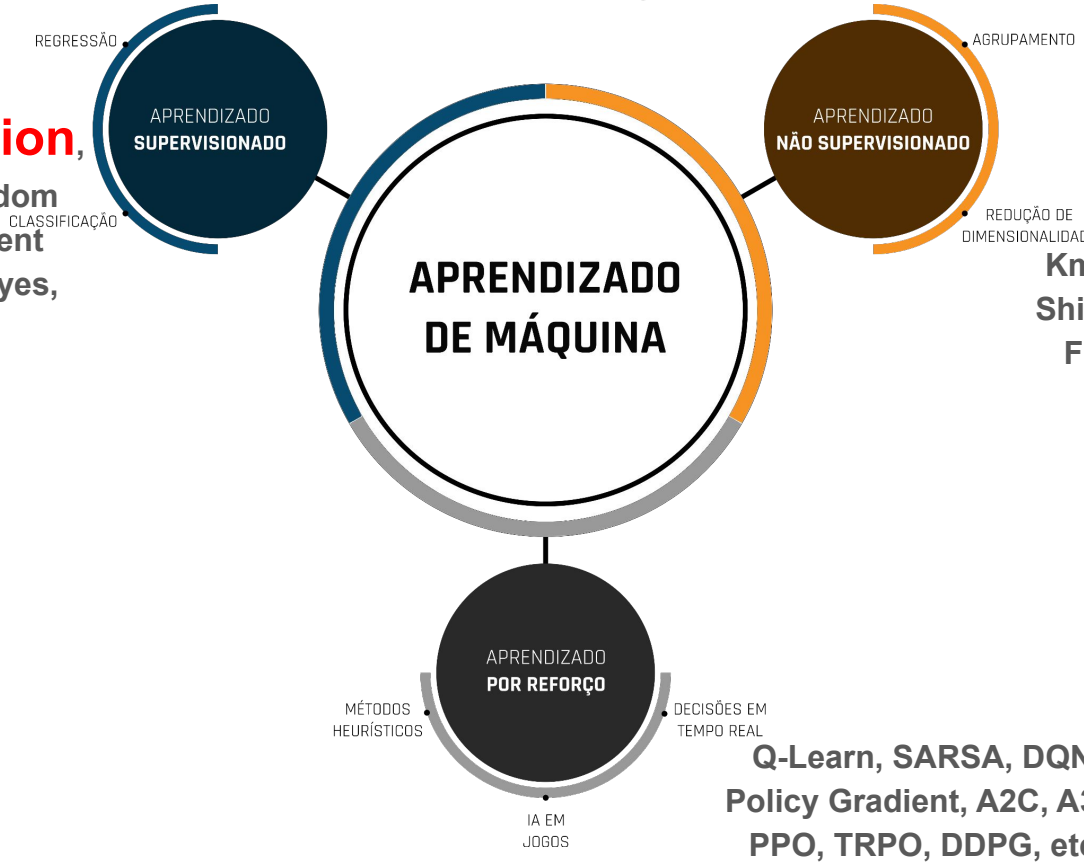
UFG
INF
INSTITUTO DE INFORMÁTICA



Aprendizado de Máquina - Paradigmas

Logistic Regression,

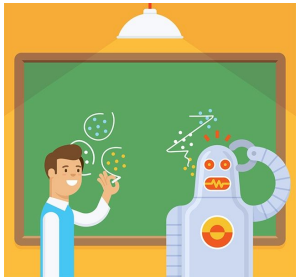
KNN, Decision Tree, Random Forest, Adaboost, Gradient Boosting, SVM, Naive Bayes, MLP, Deep Learning, Transformers, etc.



Kmeans, Dbscan, Mean Shift, PCA, t-SNE, Umap, FP-Growth, LDA, etc.

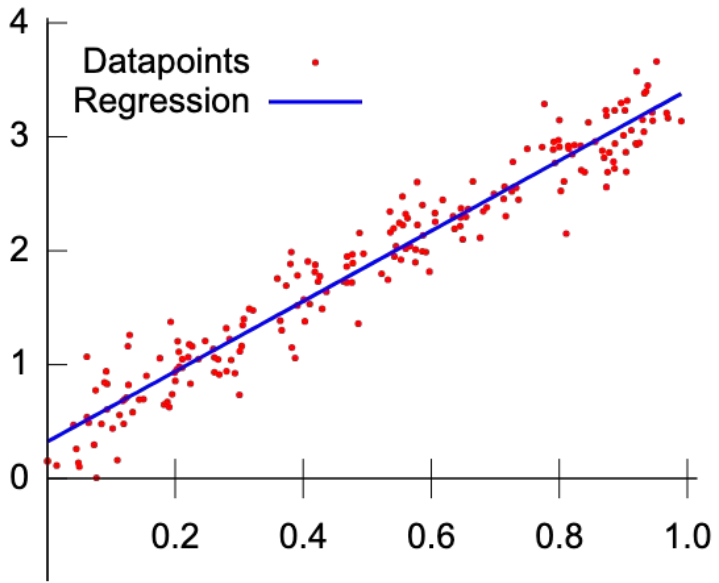
Q-Learn, SARSA, DQN, Policy Gradient, A2C, A3C, PPO, TRPO, DDPG, etc.

Aprendizado Supervisionado - Regressão



UFG INF
INSTITUTO DE INFORMÁTICA

➔ Utilizada para prever valores numéricos contínuos a partir da relação entre variáveis de entrada e uma variável de saída



Regressão Linear

- É uma técnica estatística e de aprendizado de máquina que modela a relação entre uma variável dependente e uma ou mais variáveis independentes por meio de uma reta (ou hiperplano), permitindo explicar e prever valores da variável de interesse.
  - uma variável desconhecida, ou dependente, muitas vezes chamada de variável de resultado
  - uma ou mais variáveis independentes, frequentemente chamados de preditores, covariáveis, recursos, ou features

Regressão Linear - Exemplo

- Imagine que você quer prever o preço de uma casa baseado apenas no seu tamanho em metros quadrados. A fórmula básica é a equação da reta:
  - $Y = a + b \times X$

    - Onde:
      - a → Valor de partida
      - b → Quanto o preço aumenta a cada m^2 a mais na área do imóvel
      - Y → O preço previsto do imóvel
      - X → A área do imóvel em m^2

Regressão Linear - Exemplo - regressao_01.py

- No ambiente virtual, crie uma pasta com o nome python_scripts
  - Dentro da pasta python_scripts, crie o arquivo regressao_01.py com o seguinte conteúdo:

```
import matplotlib.pyplot as plt
import numpy as np

# Valor de partida (preço base do imóvel)
a = 0

# Quanto o preço aumenta para cada m² adicional
b = 0.8

# Área do imóvel (m²)
X = np.linspace(0, 1000, 1000)

# Preço estimado do imóvel (em milhares de reais)
Y = a + b * X
```

Regressão Linear - Exemplo - regressao_01.py

- No ambiente virtual, crie uma pasta com o nome python_scripts
  - Dentro da pasta python_scripts, crie o arquivo regressao_01.py com o seguinte conteúdo:

```
# Plota a reta da regressão
plt.plot(X, Y, '-r')
# Configuração dos eixos
plt.xlim(0, 1000)
plt.ylim(0, 2000)
plt.xticks(np.arange(0, 1100, step=100))
# Rótulos
plt.xlabel('Área em $m^2$')
plt.ylabel("Preço em 1000's R$")
plt.title('Preço estimado do imóvel')
# Exibe a grade
plt.grid()
# Mostra o gráfico
plt.show()
```

Regressão Linear - Exemplo - Python Puro

- /Users/ronaldomartinsdacosta/am-env/python_scripts/regressao_01.py
- python regressao_01.py





Regressão Linear - O mesmo no Django

➔ /Users/ronaldomartinsdacosta/am-env/python_django/am/urls.py

```
from django.contrib import admin
from django.urls import path
from . import views

app_name = 'app_am'

urlpatterns = [
    path('', views.am_exemplo_01, name='am_exemplo_01'),
    path('am_exemplo_01', views.am_exemplo_01, name='am_exemplo_01'),
    path('regressao_01/', views.regressao_01, name='regressao_01'),
]
```

Regressão Linear - O mesmo no Django

➔ /Users/ronaldomartinsdacosta/am-env/python_django/am/templates/regressao_01.html

```
<html>

<head>
    <title>01_regressao</title>
</head>

<body>
    <h2>Exemplo utilizando Plotly</h2>
    {{ grafico|safe }}
</body>

</html>
```

Regressão Linear - O mesmo no Django

➔ /Users/ronaldomartinsdacosta/am-env/python_django/am/views.py

```
from django.shortcuts import render
from django.http import HttpResponse
import numpy as np
import plotly.graph_objects as go

def am_exemplo_01(request):
    return HttpResponse("Exemplos de Aprendizado de Máquina (AM)")

def regressao_01(request):
    # Valor de partida (preço base do imóvel)
    a = 0

    # Quanto o preço aumenta para cada m² adicional
    b = 0.8
```

Regressão Linear - O mesmo no Django

➔ /Users/ronaldomartinsdacosta/am-env/python_django/am/views.py

```
    # Área do imóvel (m²)
    X = np.linspace(0, 1000, 1000)

    # Preço estimado do imóvel (em milhares de reais)
    Y = a + b * X
```

Regressão Linear - O mesmo no Django

➔ /Users/ronaldomartinsdacosta/am-env/python_django/am/views.py

```
    # Cria o gráfico
    fig = go.Figure()

    fig.add_trace(go.Scatter(x=X,y=Y,mode='lines',line=dict(color='red',
width=3),name='Preço estimado'))

    # Configuração do gráfico
    fig.update_layout(title='Preço estimado do imóvel',xaxis_title='Área
(m²)',yaxis_title='Preço (1000 R$)',xaxis=dict(range=[0,
1000],tickmode='linear',dtick=100),yaxis=dict(range=[0, 2000]),template='plotly_white')
```

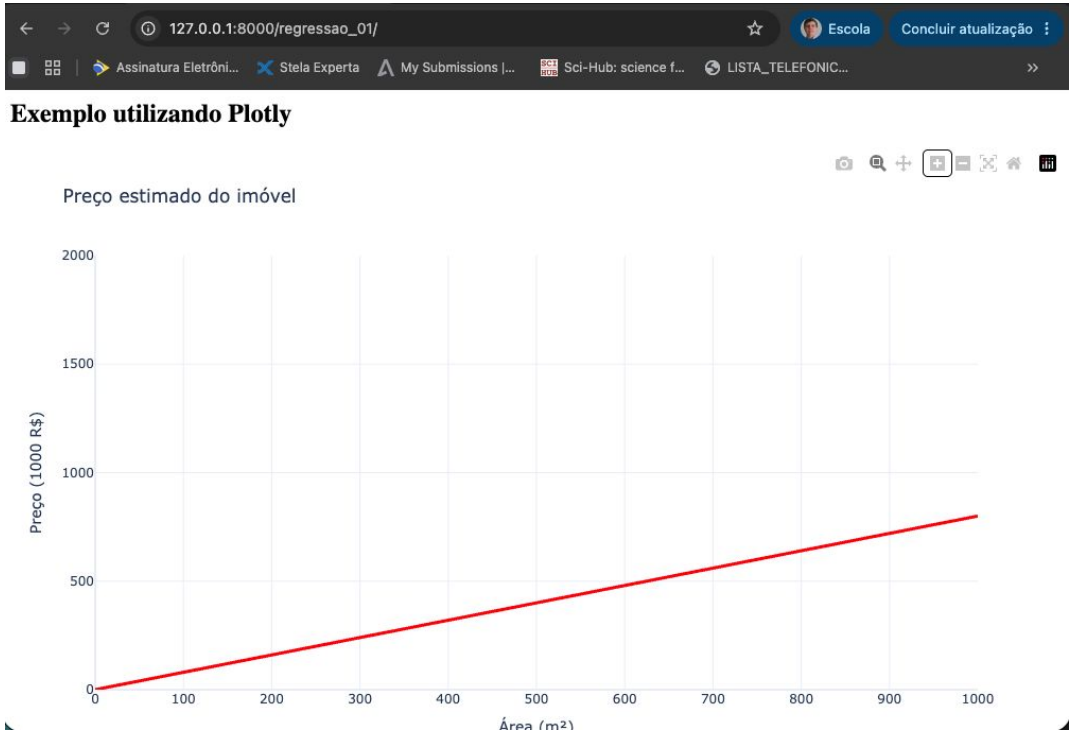
Regressão Linear - O mesmo no Django

➔ /Users/ronaldomartinsdacosta/am-env/python_django/am/views.py

```
    data = {}
    data['grafico'] = fig.to_html(full_html=False)
    return render(request, 'regressao_01.html', data)
```

Regressão Linear - O mesmo no Django

➔ http://127.0.0.1:8000/regressao_01





Regressão Linear

➔ Imaginemos que possuímos pontos de observação em que foi coletado valores reais de imóveis em função de conservação, melhorias, etc:

Imóvel	Área	Preço coletado
1	480 m²	R$ 92.000
2	510 m²	R$ 96.500
3	520 m²	R$ 98.000
4	850 m²	R$ 147.500
5	960 m²	R$ 164.000
...	...	...
11	2350 m²	R$ 372.500

Regressão Linear - Exemplo - regressao_02.py

➔ /Users/ronaldomartinsdacosta/am-env/python_scripts/regressao_02.py

```
import matplotlib.pyplot as plt
import numpy as np

# Dados observados (imóveis)
X_dados = np.array([480, 510, 520, 850, 960, 1200, 1400, 1650, 1700, 1920, 2350])
Y_dados = np.array([92, 96.5, 98, 147.5, 164, 200, 230, 267.5, 275, 308, 372.5])

# Plota os imóveis observados
plt.plot(X_dados, Y_dados, 'o', color='black')

# Parâmetros da regressão linear
a = 20      # Valor de partida
b = 0.15    # Quanto o preço aumenta para cada m² adicional

# Valores de X para desenhar a reta
X = np.linspace(0, 2500, 2500)
```

Regressão Linear - Exemplo - regressao_02.py

➔ /Users/ronaldomartinsdacosta/am-env/python_scripts/regressao_02.py

```
# Equação da regressão linear
Y = a + b * X

# Plota a reta da regressão
plt.plot(X, Y, '-r')

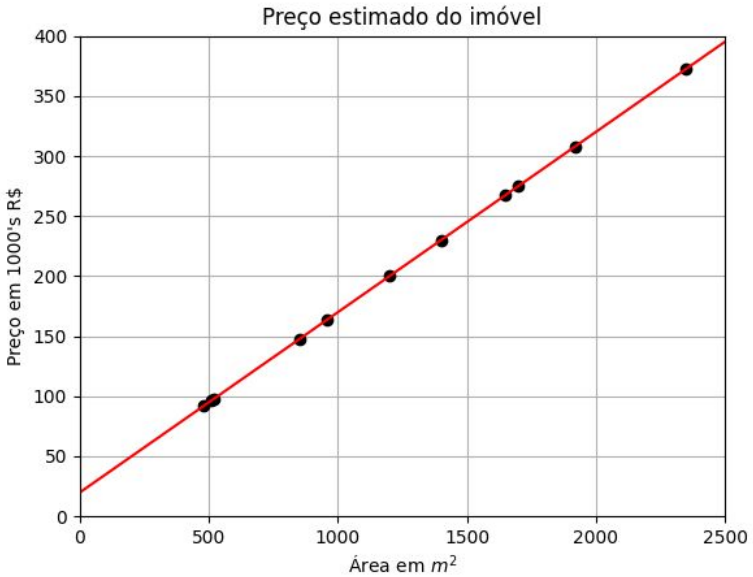
# Configuração do gráfico
plt.xlim(0, 2500)
plt.ylim(0, 400)

plt.xlabel('Área em $m^2$')
plt.ylabel("Preço em 1000's R$")
plt.title('Preço estimado do imóvel')

plt.grid()
plt.show()
```

Regressão Linear - Exemplo - Python Puro

- ➔ /Users/ronaldomartinsdacosta/am-env/python_scripts/regressao_02.py
- ➔ python regressao_02.py





Regressão Linear

- Em um problema real:
  - Na prática, você não escolhe

    a = 20 e b = 0.15

- O algoritmo de regressão linear analisa todos os imóveis do conjunto de dados e encontra automaticamente os valores de **a** e **b** que melhor ajustam a reta aos pontos observados.

Regressão Linear - regressao_03.py

➔ Encontre os valores corretos para as variáveis da equação ?

```
import matplotlib.pyplot as plt
import numpy as np
# Dados observados (lotes)
X_dados = np.array([480, 510, 520, 850, 960, 1200, 1400, 1650, 1700, 1920, 2350])
Y_dados = np.array([98, 110, 200, 210, 280, 265, 300, 287, 325, 300, 290])
# Plota os lotes observados
plt.plot(X_dados, Y_dados, 'o', color='black')
# Configuração do gráfico
plt.xlim(0, 2500)
plt.ylim(0, 400)
plt.xlabel('Área em $m^2$')
plt.ylabel("Preço em 1000's R$")
plt.title('Preço estimado do imóvel')
```

Regressão Linear - regressao_03.py

➔ Encontre os valores corretos para as variáveis da equação ?

```
#
## Ajustando manualmente os parâmetros da Regressão Linear
#
# Parâmetros da regressão linear
a = ??      # Valor de partida
b = ?.??    # Quanto o preço aumenta para cada m² adicional
# Valores de X para desenhar a reta
X = np.linspace(0, 2500, 2500)
# Equação da regressão linear
Y = a + b * X
# Plota a reta da regressão
plt.plot(X, Y, '-r')

plt.grid()
plt.show()
```

Regressão Linear - MSE

➔ Para encontrar os valores o algoritmos de Regressão Linear utiliza o *Mean Square Error* (MSE - Erro Médio Quadrático)

$$\text{MSE} = \frac{1}{n}\sum_{i=1}^{n}(Y_i - \hat{Y}_i)^2$$

Onde:

$n \rightarrow$ Quantidade de valores

$Y_i \rightarrow$ Valores Observados

$\hat{Y}_i \rightarrow$ Valores Preditos

Calculando o MSE

Área (X)	Área ao quadrado (X^2)	Preço (Y)	Área x Preço	a	b	Previsão (a+bX)	Erro (Preço - Previsão)	$Erro^2$
480	230400	98	47040	120,23986	0,09914044	167,83	-69,83	4876,2289
510	260100	110	56100	120,23986	0,09914044	170,8	-60,8	3696,64
520	270400	200	104000	120,23986	0,09914044	171,79	28,21	795,8041
850	722500	210	178500	120,23986	0,09914044	204,51	5,49	30,1401
960	921600	280	268800	120,23986	0,09914044	215,42	64,58	4170,5764
1200	1440000	265	318000	120,23986	0,09914044	239,21	25,79	665,1241
1400	1960000	300	420000	120,23986	0,09914044	259,03	40,97	1678,5409
1650	2722500	287	473550	120,23986	0,09914044	283,87	3,13	9,7969
1700	2890000	325	552500	120,23986	0,09914044	288,82	36,18	1308,9924
1920	3686400	300	576000	120,23986	0,09914044	310,63	-10,63	112,9969
2350	5522500	290	681500	120,23986	0,09914044	353,26	-63,26	4001,8276
13540	**20626400**	**2665**	**3675990**					**21346,6683**

$$b = \frac{(11 \times 3675990) - (13540 \times 2665)}{(11 \times 20625400) - 13540^2}$$

$$b = \frac{430779}{43452100}$$

$$b = 0.09914044$$

$$a = \frac{2665 - (0{,}09914044 \times 13540)}{11}$$

$$a = \frac{1322{,}6384}{11}$$

$$a = 120{,}23986$$

$$MSE = \frac{21346{,}6683}{11}$$

$$MSE = 1940{,}61$$

Regressão Linear com MSE - regressao_04.py

```
import matplotlib.pyplot as plt
import numpy as np
# Dados observados (lotes)
X_dados = np.array([480, 510, 520, 850, 960, 1200, 1400, 1650, 1700, 1920, 2350])
Y_dados = np.array([98, 110, 200, 210, 280, 265, 300, 287, 325, 300, 290])
# ----------------------------------------------------------------
# Procura pelos melhores parâmetros utilizando o Mean Squared Error
# ----------------------------------------------------------------
menor_mse = np.inf
print("Valor Inicial para MSE:", menor_mse)
melhor_a = 0
melhor_b = 0
# Inclinação aproximada
b_estimado = (Y_dados.max() - Y_dados.min()) / (X_dados.max() - X_dados.min())
# Intercepto aproximado
a_estimado = np.mean(Y_dados - b_estimado * X_dados)
print("Estimado para a:", a_estimado)
print("Estimado para b:", b_estimado)
```

Regressão Linear com MSE - regressao_04.py

```
# Valores candidatos para a e b
for a in np.arange(a_estimado - 100, a_estimado + 100, 1):
    for b in np.arange(b_estimado - 0.10, b_estimado + 0.10, 0.001):
        # Preços previstos
        Y_predito = a + b * X_dados

        # Mean Squared Error
        mse = np.mean((Y_dados - Y_predito) ** 2)

        # Guarda os melhores parâmetros
        if mse < menor_mse:
            menor_mse = mse
            melhor_a = a
            melhor_b = b

print(f"Melhor valor de a: {melhor_a:.2f}")
print(f"Melhor valor de b: {melhor_b:.4f}")
print(f"MSE: {menor_mse:.2f}")
```

Regressão Linear com MSE - regressao_04.py

```
# ------------------------------------------------------------------
# Plota os dados
# ------------------------------------------------------------------
plt.plot(X_dados, Y_dados, 'o', color='black', label='Dados observados')
# Reta encontrada
X = np.linspace(0, 2500, 2500)
Y = melhor_a + melhor_b * X
plt.plot(X, Y, '-r', linewidth=2, label='Melhor reta')
plt.xlim(0, 2500)
plt.ylim(0, 400)
plt.xlabel('Área em $m^2$')
plt.ylabel("Preço em 1000's R$")
plt.title('Regressão Linear utilizando MSE')

plt.grid()
plt.legend()

plt.show()
```

Regressão Linear com Sckilearn
regressao_05.py

```
pip install scikit-learn
```

```
import matplotlib.pyplot as plt
import numpy as np

from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error

# ------------------------------------------------------------
# Dados observados
# ------------------------------------------------------------
X_dados = np.array([480, 510, 520, 850, 960, 1200, 1400, 1650, 1700, 1920, 2350])
Y_dados = np.array([98, 110, 200, 210, 280, 265, 300, 287, 325, 300, 290])

# O Scikit-Learn exige uma matriz (n_amostras, n_variáveis)
X = X_dados.reshape(-1, 1)
```

Regressão Linear com Sckilearn
regressao_05.py

```
# ------------------------------------------------------------
# Criação do modelo
# ------------------------------------------------------------
modelo = LinearRegression()

# Treinamento
modelo.fit(X, Y_dados)

# ------------------------------------------------------------
# Coeficientes da reta
# ------------------------------------------------------------
a = modelo.intercept_
b = modelo.coef_[0]

print(f"Intercepto (a): {a:.4f}")
print(f"Inclinação (b): {b:.6f}")
```

Regressão Linear com Sckilearn
regressao_05.py

```
# ------------------------------------------------------------
# Previsões
# ------------------------------------------------------------
Y_predito = modelo.predict(X)

# ------------------------------------------------------------
# Mean Squared Error
# ------------------------------------------------------------

mse = mean_squared_error(Y_dados, Y_predito)
print(f"MSE: {mse:.4f}")

# ------------------------------------------------------------
# Equação da reta
# ------------------------------------------------------------
print(f"\nEquação da reta:")
print(f"Preço = {a:.4f} + {b:.6f} × Área")
```

Regressão Linear com Sckilearn
regressao_05.py

```
# ------------------------------------------------------------
# Gráfico
# ------------------------------------------------------------
plt.figure(figsize=(8,5))
plt.scatter(X_dados, Y_dados, color="black", label="Dados observados")

X_linha = np.linspace(0, 2500, 300).reshape(-1,1)
Y_linha = modelo.predict(X_linha)

plt.plot(X_linha, Y_linha, color="red", linewidth=2, label="Regressão Linear")
plt.xlim(0,2500)
plt.ylim(0,400)
plt.xlabel("Área em m²")
plt.ylabel("Preço (1000's R$)")
plt.title("Regressão Linear utilizando Scikit-Learn")
plt.grid(True)
plt.legend()
plt.show()
```

Regressão Linear com Sckilearn
regressao_06.py

```
pip install pandas
```

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error
# ------------------------------------------------------------
# Leitura do arquivo CSV
# ------------------------------------------------------------
df = pd.read_csv("regressao_06.csv")

# Remove espaços dos nomes das colunas
df.columns = df.columns.str.strip()
```

Regressão Linear com Sckilearn
regressao_06.py

```
# ------------------------------------------------------------
# Dados observados
# X = Life expectancy
# Y = Adult Mortality
# ------------------------------------------------------------
X_dados = df["Life expectancy"].values
Y_dados = df["Adult Mortality"].values

# Remove linhas com valores ausentes
dados = pd.DataFrame({
    "X": X_dados,
    "Y": Y_dados
}).dropna()

X_dados = dados["X"].values
Y_dados = dados["Y"].values

# O Scikit-Learn exige uma matriz (n_amostras, n_variáveis)
X = X_dados.reshape(-1, 1)
```

Regressão Linear com Sckilearn
regressao_06.py

```
# ------------------------------------------------------------
# Criação do modelo
# ------------------------------------------------------------
modelo = LinearRegression()

# Treinamento
modelo.fit(X, Y_dados)

# ------------------------------------------------------------
# Coeficientes da reta
# ------------------------------------------------------------
a = modelo.intercept_
b = modelo.coef_[0]

print(f"Intercepto (a): {a:.4f}")
print(f"Inclinação (b): {b:.6f}")
```

Regressão Linear com Sckilearn
regressao_06.py

```
# ------------------------------------------------------------
# Previsões
# ------------------------------------------------------------
Y_predito = modelo.predict(X)

# ------------------------------------------------------------
# Mean Squared Error
# ------------------------------------------------------------
mse = mean_squared_error(Y_dados, Y_predito)

print(f"MSE: {mse:.4f}")

# ------------------------------------------------------------
# Equação da reta
# ------------------------------------------------------------
print("\nEquação da reta:")
print(f"Adult Mortality = {a:.4f} + {b:.6f} × Life expectancy")
```

Regressão Linear com Sckilearn
regressao_06.py

```
# ------------------------------------------------------------
# Gráfico
# ------------------------------------------------------------
plt.figure(figsize=(8,5))

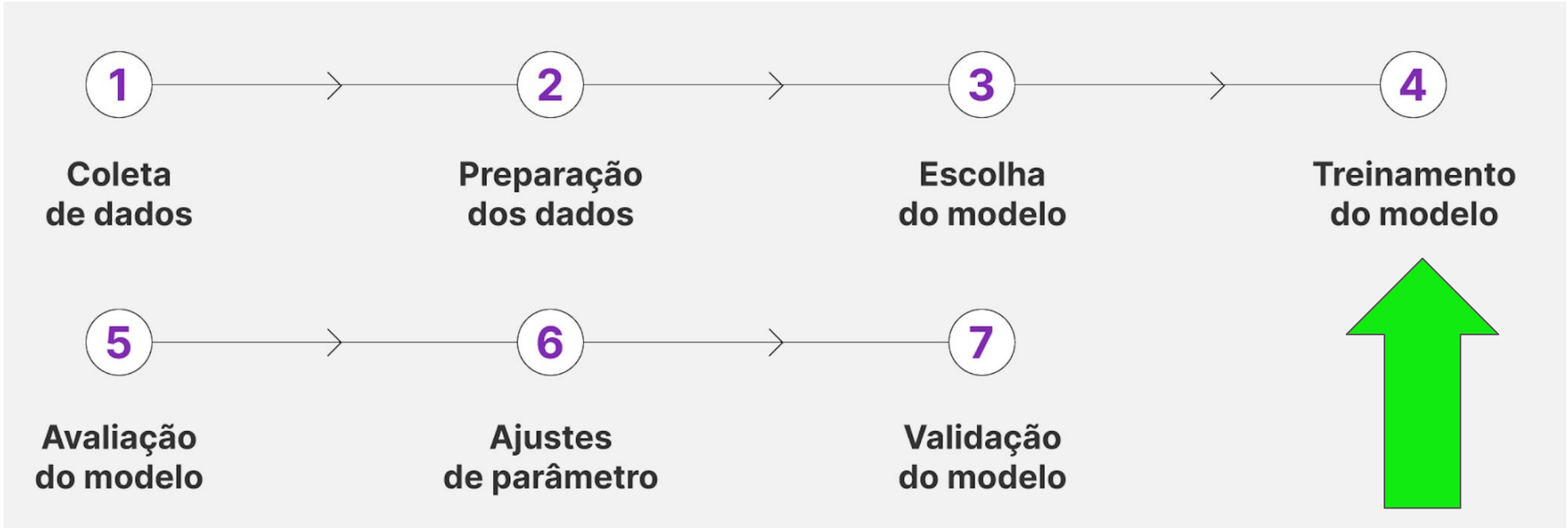
plt.scatter(X_dados, Y_dados, color="black", label="Dados observados")

X_linha = np.linspace(X_dados.min(),X_dados.max(),300).reshape(-1,1)
Y_linha = modelo.predict(X_linha)

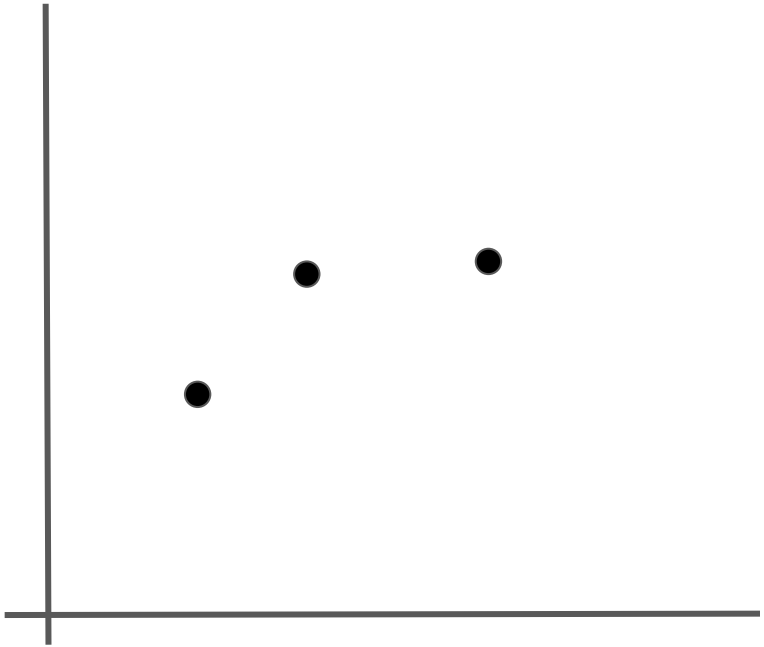
plt.plot(X_linha,Y_linha,color="red",linewidth=2,label="Regressão Linear")
plt.xlabel("Life expectancy")
plt.ylabel("Adult Mortality")
plt.title("Regressão Linear utilizando Scikit-Learn")
plt.grid(True)
plt.legend()

plt.show()
```

Treinamento de Modelos

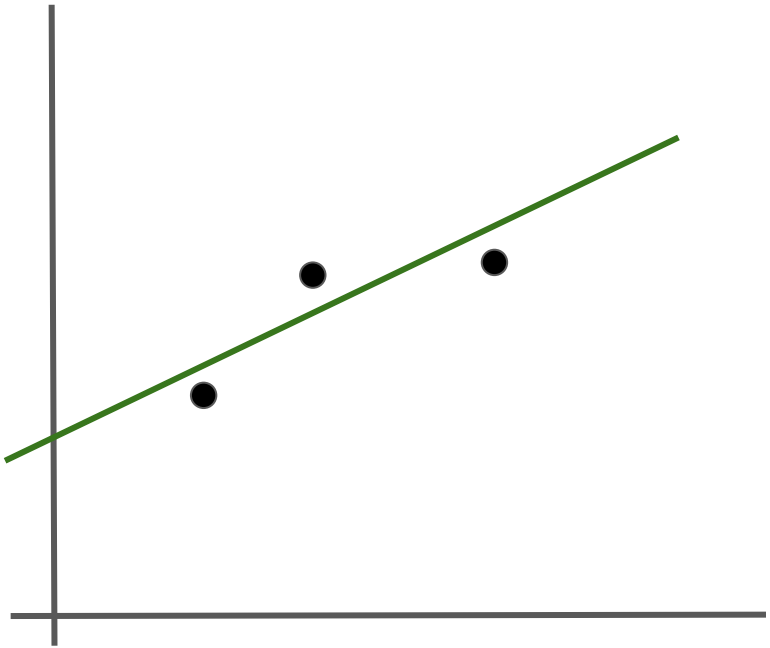


Treinamento de Modelos



➔ Existem estes pontos observados e desejamos encontrar a melhor reta que passe por eles

Treinamento de Modelos



➔ O processo não é tão simples e pode demandar várias tentativas

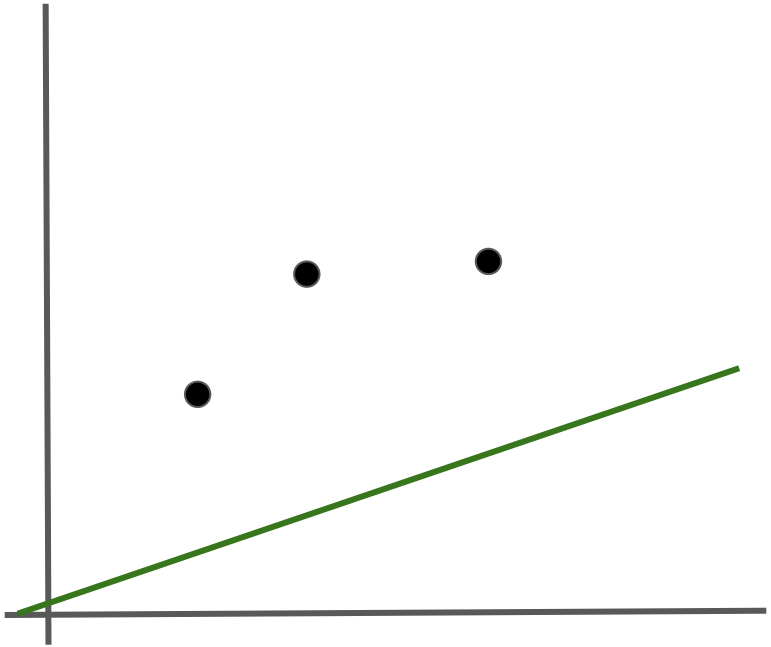
Treinamento de Modelos

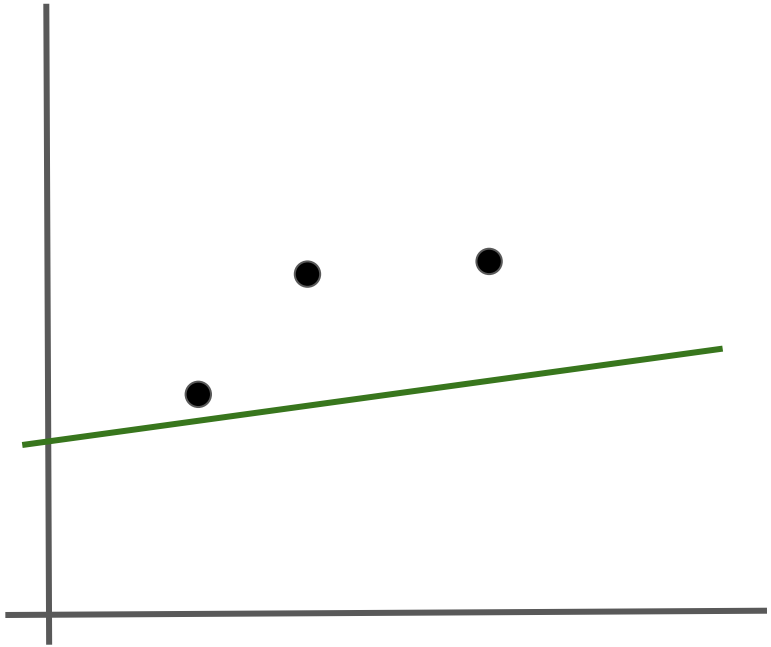


UFG INF
INSTITUTO DE INFORMÁTICA



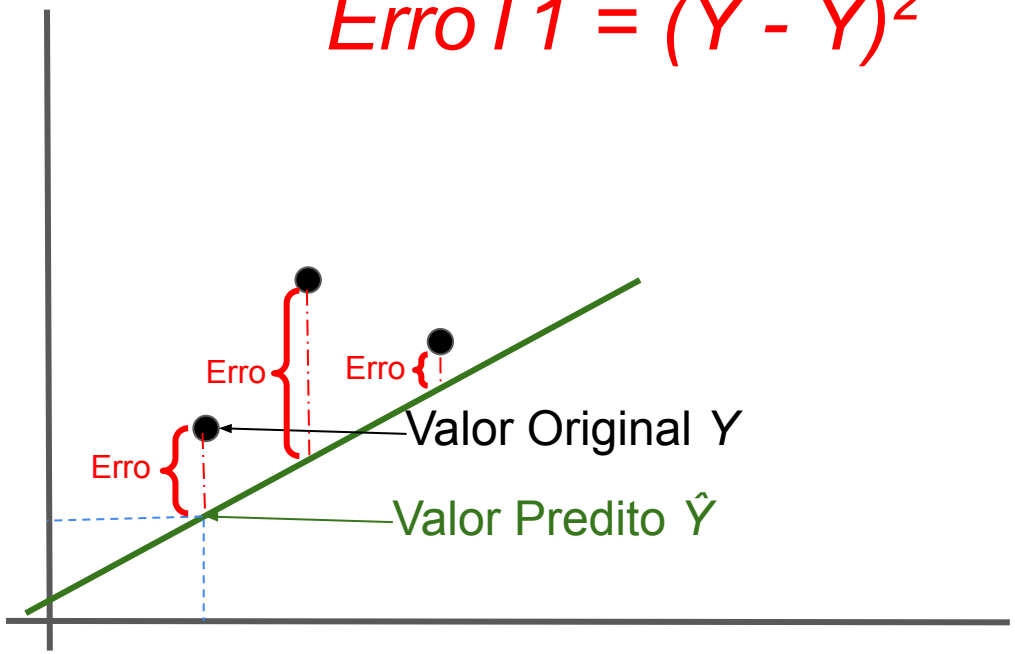
➔ O processo não é tão simples e pode demandar várias tentativas





Erro Médio Quadrático

$$ErroT1 = (Y - \hat{Y})^2$$

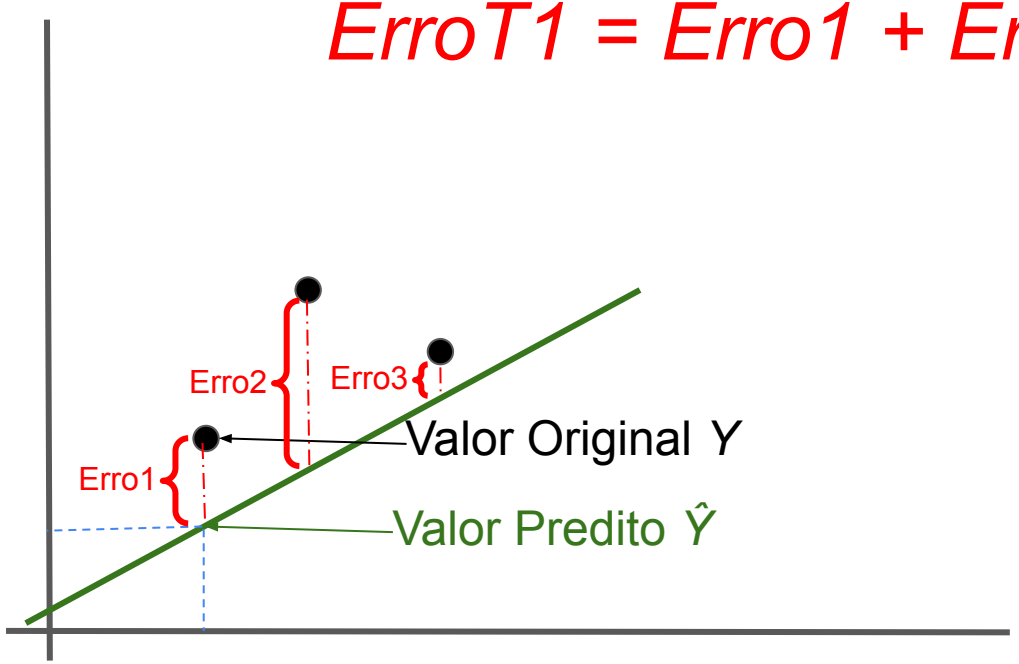


Erro Médio Quadrático

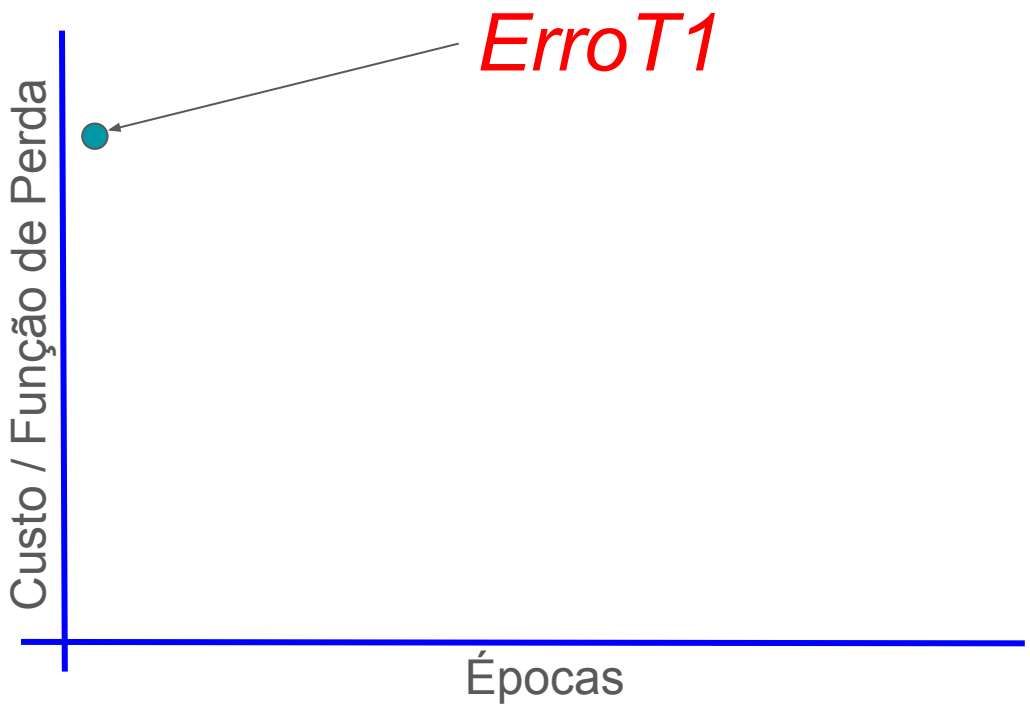


UFG INF
INSTITUTO DE INFORMÁTICA

$$ErroT1 = Erro1 + Erro2 + Erro3$$

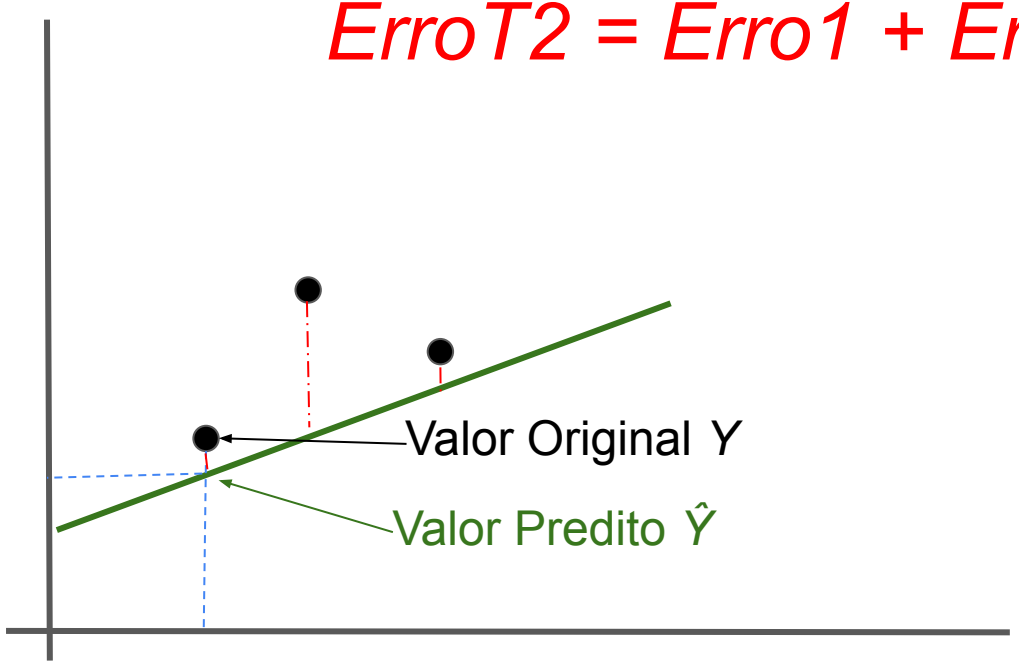


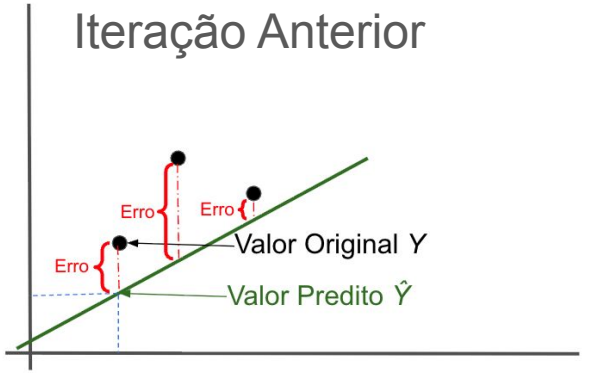
Erro Médio Quadrático



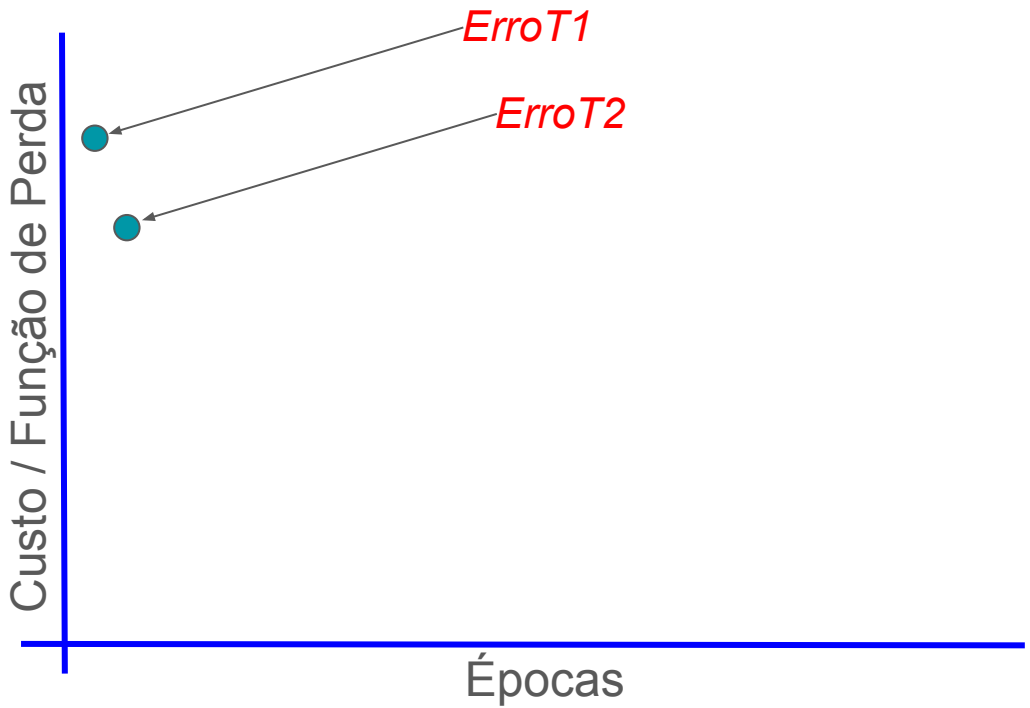
Erro Médio Quadrático

$$ErroT2 = Erro1 + Erro2 + Erro3$$

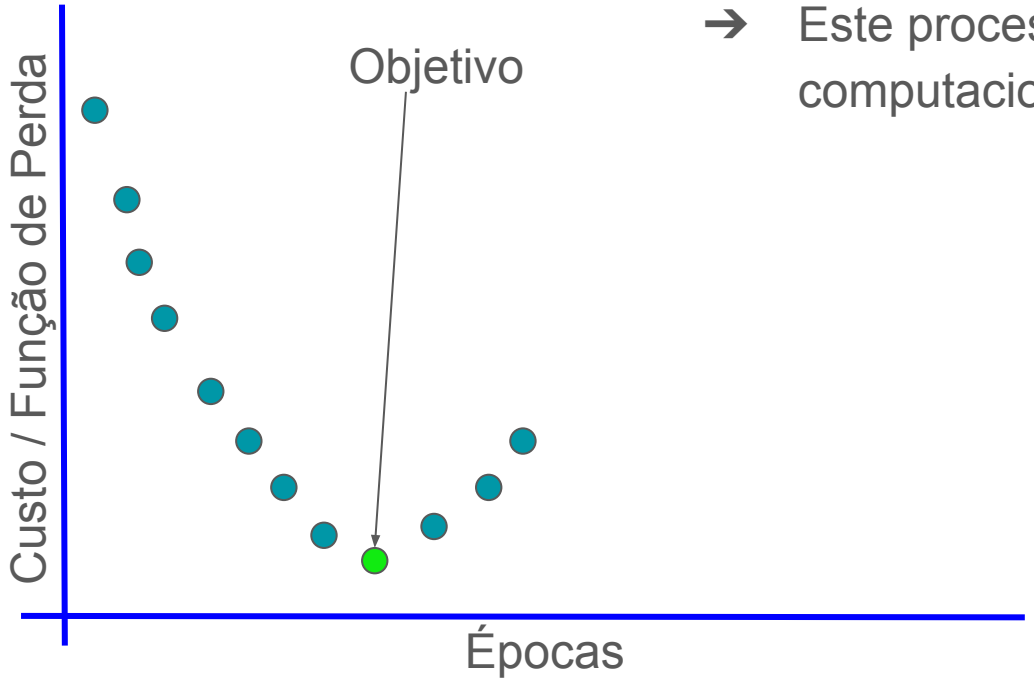




Erro Médio Quadrático

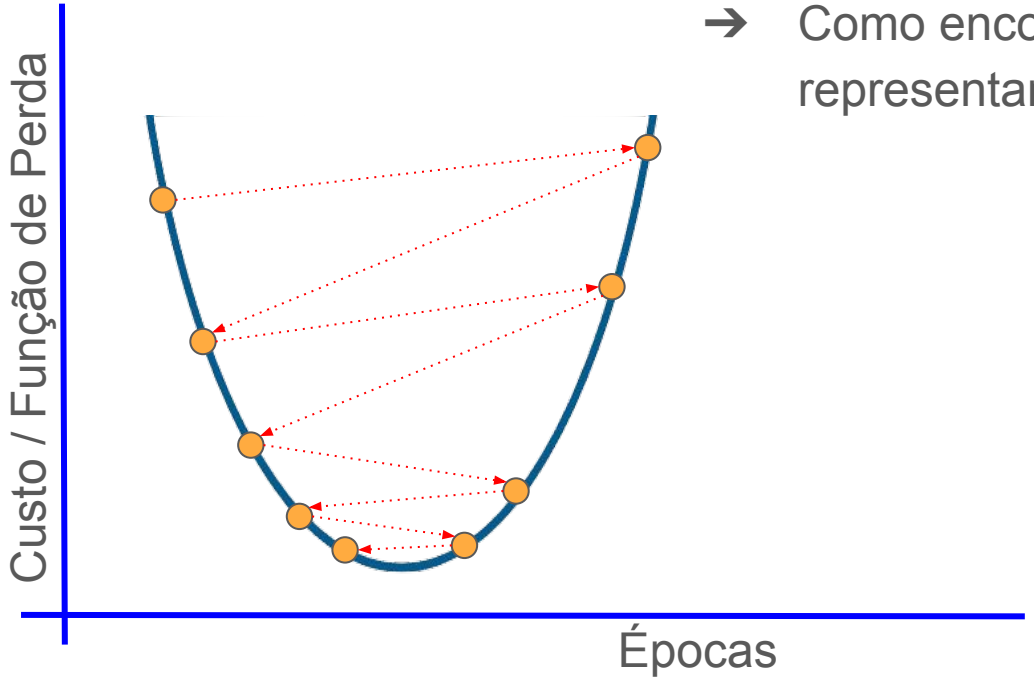


Erro Médio Quadrático



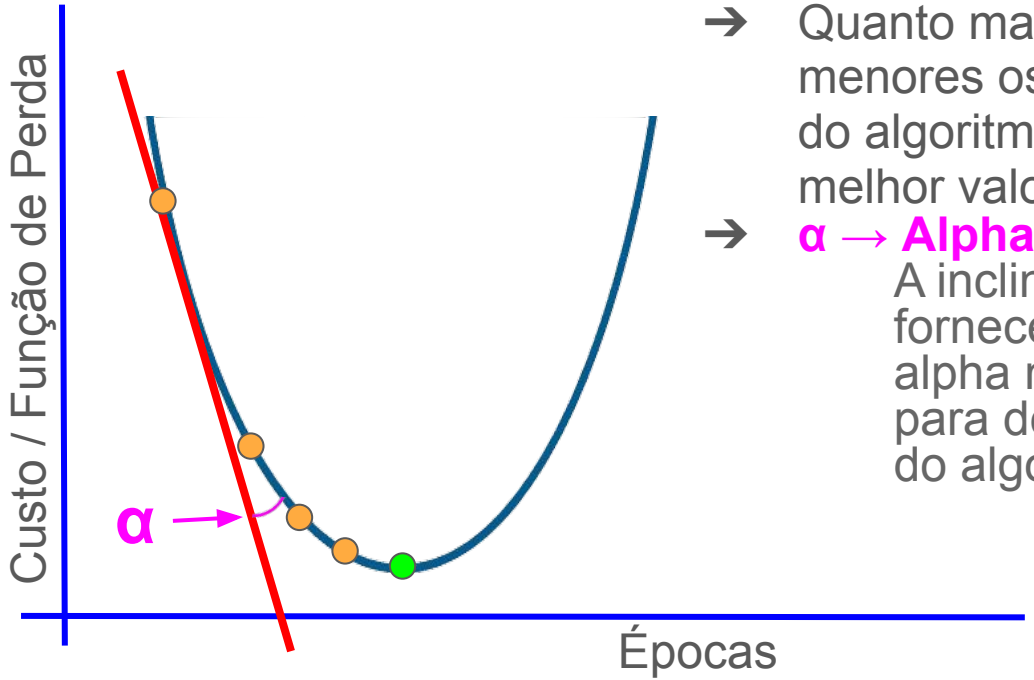
➔ Este processo possui alto custo computacional

Gradiente Descendente



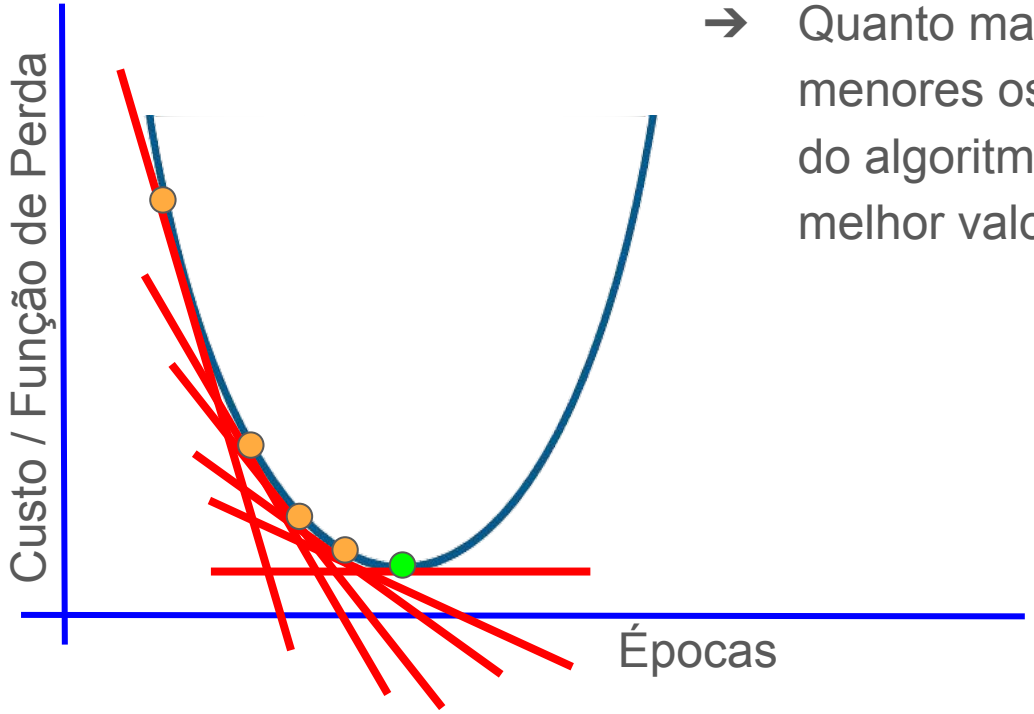
➔ Como encontrar os valores que representam o menor custo ?

Gradiente Descendente



- ➔ Quanto mais próximo do valor ideal, menores os passos entre as iterações do algoritmo, tendendo a encontrar o melhor valor
- ➔ **α → Alpha**
  A inclinação da reta tangente fornece a derivada (gradiente). O alpha multiplica esse gradiente para definir o tamanho do passo do algoritmo

Gradiente Descendente



➔ Quanto mais próximo do valor ideal, menores os passos entre as iterações do algoritmo, tendendo a encontrar o melhor valor

Regressão Linear - Gradiente Descendente
regressao_07.py

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error
# ============================================================
# DADOS
# ============================================================
X = np.array([480, 510, 520, 850, 960, 1200, 1400, 1650, 1700, 1920, 2350],
dtype=float)
Y = np.array([98,  110, 200, 210, 280,  265,  300,  287,  325,  300,  290],
dtype=float)
m = len(X)
# ============================================================
# NORMALIZAÇÃO
# (Facilita o Gradiente Descendente)
# ============================================================
media = np.mean(X)
desvio = np.std(X)
Xn = (X - media) / desvio
```

Regressão Linear - Gradiente Descendente
regressao_07.py

```
# ============================================================
# PARÂMETROS INICIAIS
# ============================================================
a = 0.0
b = 0.0
taxa = 0.05
epocas = 1000
historico_custo = []
# ============================================================
# GRADIENTE DESCENDENTE
# ============================================================
for epoca in range(epocas):
    # previsão
    Y_pred = a + b * Xn
    # erro
    erro = Y_pred - Y
    # função de custo
    J = np.sum(erro ** 2) / (2 * m)
    historico_custo.append(J)
```

Regressão Linear - Gradiente Descendente
regressao_07.py

```
    # gradientes
    da = np.sum(erro) / m
    db = np.sum(erro * Xn) / m
    # atualização
    a = a - taxa * da
    b = b - taxa * db
# ============================================================
# RESULTADO
# ============================================================
print("\n==============================")
print("GRADIENTE DESCENDENTE")
print("==============================")
print(f"Intercepto (normalizado): {a:.6f}")
print(f"Inclinação (normalizada): {b:.6f}")
# ============================================================
# PREVISÕES
# ============================================================
Y_pred = a + b * Xn
mse = mean_squared_error(Y, Y_pred)
```

Regressão Linear - Gradiente Descendente
regressao_07.py

```
print(f"MSE = {mse:.6f}")
# ============================================================
# COMPARAÇÃO COM SCIKIT-LEARN
# ============================================================
modelo = LinearRegression()
modelo.fit(X.reshape(-1,1),Y)
print("\n==============================")
print("SCIKIT-LEARN")
print("==============================")
print(f"Intercepto = {modelo.intercept_:.6f}")
print(f"Inclinação = {modelo.coef_[0]:.6f}")
print(f"MSE = {mean_squared_error(Y,modelo.predict(X.reshape(-1,1))):.6f}")
# ============================================================
# RETA
# ============================================================
x_plot = np.linspace(min(X),max(X),300)
x_plot_norm = (x_plot-media)/desvio
y_plot = a + b*x_plot_norm
```

Regressão Linear - Gradiente Descendente
regressao_07.py

```
# ============================================================
# GRÁFICO 1
# ============================================================
plt.figure(figsize=(10,6))
plt.scatter(X,Y,color="black",s=70,label="Dados")
plt.plot(x_plot,y_plot,color="red",linewidth=3,label="Gradiente Descendente")
plt.xlabel("Área (m²)")
plt.ylabel("Preço (1000's R$)")
plt.title("Regressão Linear - Gradiente Descendente")
plt.grid(True)
plt.legend()
plt.show()
# ============================================================
# GRÁFICO DA FUNÇÃO DE CUSTO
# ============================================================
plt.figure(figsize=(10,5))
plt.plot(historico_custo,linewidth=2)
plt.xlabel("Época")
plt.ylabel("Função de Custo J")
```

Regressão Linear - Gradiente Descendente
regressao_07.py

```
plt.title("Convergência do Gradiente Descendente")
plt.grid(True)
plt.show()
# ============================================================
# CONSULTA
# ============================================================
print("\n==============================")
print("Estimativa de Preço")
print("Digite 0 para sair.")
print("==============================")
while True:
    area = float(input("\nÁrea (m²): "))
    if area == 0:
        break
    area_norm = (area-media)/desvio
    preco = a + b*area_norm
    print(f"Preço estimado = R$ {preco*1000:,.2f}")
```

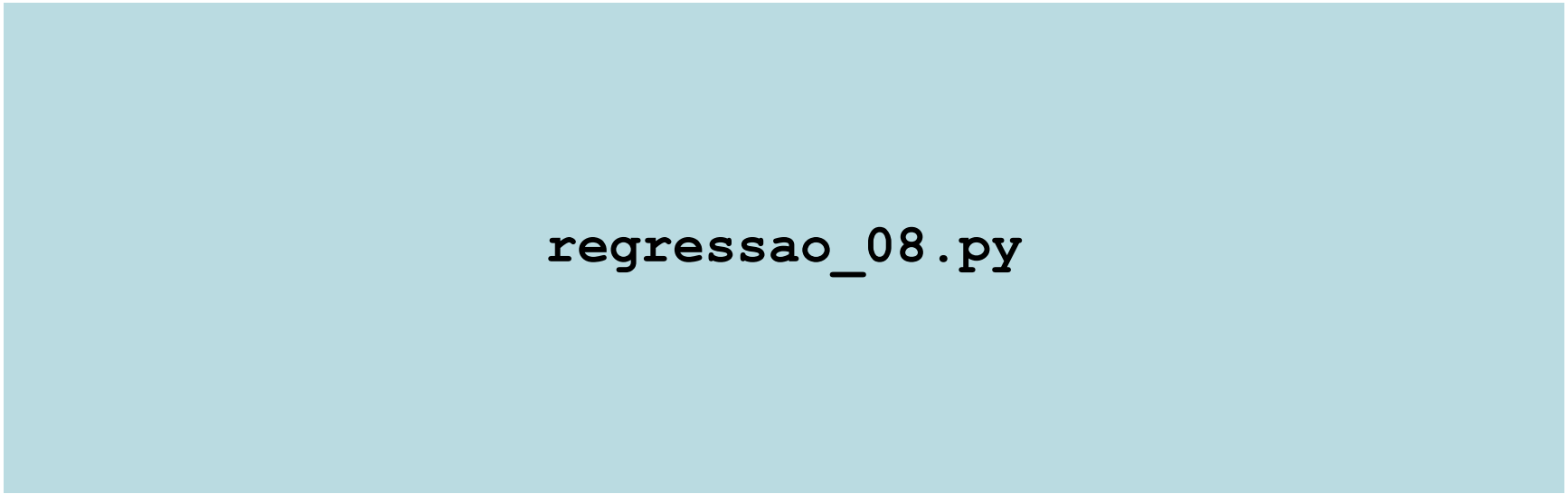
Regressão Polinomial

➔ É uma extensão da regressão linear que permite modelar relações não lineares entre as variáveis

$$Y = a + b_1 X + b_2 X^2$$

Característica	Regressão Linear	Regressão Polinomial
Forma da curva	Reta	Curva
Equação	$Y=a+bX$	$Y=a+b_1X+b_2X^2+\cdots$
Relação entre as variáveis	Linear	Pode representar relações não lineares
Complexidade	Baixa	Maior, depende do grau do polinômio
Número de coeficientes	2	Grau + 1
Interpretação	Mais simples	Mais complexa

Regressão Polinomial
regressao_08.py



Regressão Linear Múltipla

- A Regressão Linear Múltipla é um algoritmo de aprendizado supervisionado utilizado para prever o valor de uma variável dependente a partir de duas ou mais variáveis independentes

- Enquanto a regressão linear simples utiliza apenas uma variável de entrada, a regressão linear múltipla utiliza várias variáveis ao mesmo tempo, tornando a previsão geralmente mais precisa

Regressão Linear Múltipla

➔ Imagine que desejamos prever a expectativa de vida (Life Expectancy) de um país

Sabemos que ela depende de diversos fatores, por exemplo:

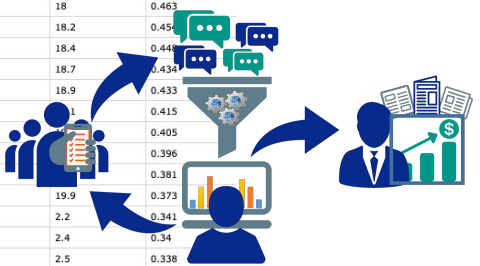
- Mortalidade de adultos (Adult Mortality)
- Consumo de álcool (Alcohol)
- Magreza entre 1 e 19 anos (Thinness 1–19 years)
- Escolaridade (Schooling)

➔ Em vez de utilizar apenas uma dessas informações, utilizamos todas simultaneamente

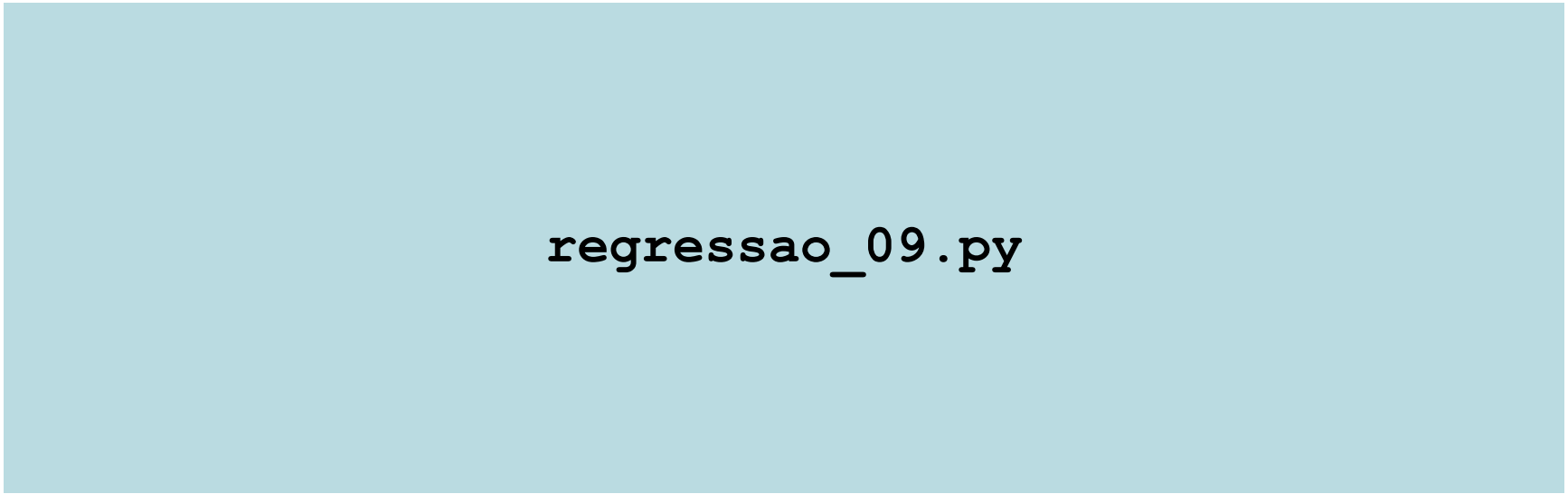
Regressão Linear Múltipla - Conhecer os Dados!

UFG INF
INSTITUTO DE INFORMÁTICA

Country	Year	Status	Life expectancy	Adult Mortality	infant deaths	Alcohol	percentage expenditure	Hepatitis B	Measles	BMI	under-five deaths	Polio	Total expenditure	Diphtheria	HIV/AIDS	GDP	Population	thinness 1-19 years	thinness 5-9 years	Income compositio
Afghanistan	2015	Developing	65	263	62	0.01	71.27962362	65	1154	19.1	83	6	8.16	65	0.1	584.25921	33736494	17.2	17.3	0.479
Afghanistan	2014	Developing	59.9	271	64	0.01	73.52358168	62	492	18.6	86	58	8.18	62	0.1	612.696514	327582	17.5	17.5	0.476
Afghanistan	2013	Developing	59.9	268	66	0.01	73.21924272	64	430	18.1	89	62	8.13	64	0.1	631.744976	31731688	17.7	17.7	0.47
Afghanistan	2012	Developing	59.5	272	69	0.01	78.1842153	67	2787	17.6	93	67	8.52	67	0.1	669.959	3696958	17.9	18	0.463
Afghanistan	2011	Developing	59.2	275	71	0.01	7.097108703	68	3013	17.2	97	68	7.87	68	0.1	63.537231	2978599	18.2	18.2	0.45
Afghanistan	2010	Developing	58.8	279	74	0.01	79.67936736	66	1989	16.7	102	66	9.2	66	0.1	553.32894	2883167	18.4	18.4	0.44
Afghanistan	2009	Developing	58.6	281	77	0.01	56.76221682	63	2861	16.2	106	63	9.42	63	0.1	445.8932979	284331	18.6	18.7	0.434
Afghanistan	2008	Developing	58.1	287	80	0.03	25.87392536	64	1599	15.7	110	64	8.33	64	0.1	373.3611163	2729431	18.8	18.9	0.433
Afghanistan	2007	Developing	57.5	295	82	0.02	10.91015598	63	1141	15.2	113	63	6.73	63	0.1	369.835796	26616792	19	1	0.415
Afghanistan	2006	Developing	57.3	295	84	0.03	17.17151751	64	1990	14.7	116	58	7.43	58	0.1	272.56377	2589345	19.2	19.2	0.405
Afghanistan	2005	Developing	57.3	291	85	0.02	1.388647732	66	1296	14.2	118	58	8.7	58	0.1	25.2941299	257798	19.3	19.3	0.396
Afghanistan	2004	Developing	57	293	87	0.02	15.29606643	67	466	13.8	120	5	8.79	5	0.1	219.1413528	24118979	19.5	19.5	0.381
Afghanistan	2003	Developing	56.7	295	87	0.01	11.08905273	65	798	13.4	122	41	8.82	41	0.1	198.7285436	2364851	19.7	19.9	0.373
Afghanistan	2002	Developing	56.2	3	88	0.01	16.88735091	64	2486	13	122	36	7.76	36	0.1	187.84595	21979923	19.9	2.2	0.341
Afghanistan	2001	Developing	55.3	316	88	0.01	10.5747282	63	8762	12.6	122	35	7.8	33	0.1	117.49698	2966463	2.1	2.4	0.34
Afghanistan	2000	Developing	54.8	321	88	0.01	10.42496	62	6532	12.2	122	24	8.2	24	0.1	114.56	293756	2.3	2.5	0.338
Albania	2015	Developing	77.8	74	0	4.6	364.9752287	99	0	58	0	99	6	99	0.1	3954.22783	28873	1.2	1.3	0.762
Albania	2014	Developing	77.5	8	0	4.51	428.7490668	98	0	57.2	1	98	5.88	98	0.1	4575.763787	288914	1.2	1.3	0.761
Albania	2013	Developing	77.2	84	0	4.76	430.8769785	99	0	56.5	1	99	5.66	99	0.1	4414.72314	289592	1.3	1.4	0.759
Albania	2012	Developing	76.9	86	0	5.14	412.4433563	99	9	55.8	1	99	5.59	99	0.1	4247.61438	2941	1.3	1.4	0.752
Albania	2011	Developing	76.6	88	0	5.37	437.0621	99	28	55.1	1	99	5.71	99	0.1	4437.17868	295195	1.4	1.5	0.738
Albania	2010	Developing	76.2	91	1	5.28	41.82275719	99	10	54.3	1	99	5.34	99	0.1	494.358832	291321	1.4	1.5	0.725
Albania	2009	Developing	76.1	91	1	5.79	348.0559517	98	0	53.5	1	98	5.79	98	0.1	4114.136545	2927519	1.5	1.6	0.721
Albania	2008	Developing	75.3	1	1	5.61	36.62206845	99	0	52.6	1	99	5.87	99	0.1	437.539647	2947314	1.6	1.6	0.713
Albania	2007	Developing	75.9	9	1	5.58	32.24655228	98	22	51.7	1	99	6.1	98	0.1	363.13685	29717	1.6	1.7	0.703
Albania	2006	Developing	74.2	99	1	5.31	3.3021542	98	68	5.8	1	97	5.86	97	0.1	35.1293	2992547	1.7	1.8	0.696
Albania	2005	Developing	73.5	15	1	5.16	26.99312143	98	6	49.9	1	97	6.12	98	0.1	279.142931	311487	1.8	1.8	0.685
Albania	2004	Developing	73	17	1	4.54	221.8428	99	7	48.9	1	98	6.38	97	0.1	2416.588235	326939	1.8	1.9	0.681
Albania	2003	Developing	72.8	18	1	4.29	14.71928882	97	8	47.9	1	97	6.27	97	0.1	189.681557	339616	1.9	2	0.674
Albania	2002	Developing	73.3	15	1	3.73	104.5169157	96	16	46.9	1	98	6.3	98	0.1	1453.642777	3511	2	2.1	0.67
Albania	2001	Developing	73.6	14	1	4.25	96.20557078	96	18	46	1	97	6	97	0.1	1326.97339	36173	2.1	2.1	0.662
Albania	2000	Developing	72.6	11	1	3.66	91.71154052	96	662	45	1	97	6.26	97	0.1	1175.788981	38927	2.1	2.2	0.656
Algeria	2015	Developing	75.6	19	21		0	95	63	59.5	24	95		95	0.1	4132.76292	39871528	6	5.8	0.743
Algeria	2014	Developing	75.4	11	21	0.01	54.2373183	95	0	58.4	24	95	7.21	95	0.1	547.8517	39113313	6	5.8	0.741
Algeria	2013	Developing	75.3	112	21	0.53	544.4507432	95	25	57.2	24	95	7.12	95	0.1	5471.866766	38338562	5.9	5.8	0.737
Algeria	2012	Developing	75.1	113	21	0.66	555.9260834	95	18	56.1	24	95	6.14	95	0.1	5564.82566	37565847	5.9	5.8	0.732
Algeria	2011	Developing	74.9	116	21	0.56	509.0020405	95	112	55	24	95	5.29	95	0.1	5432.2523	36819558	5.9	5.8	0.724
Algeria	2010	Developing	74.7	119	21	0.45	430.7175861	95	103	53.9	24	95	5.12	95	0.1	4463.394675	36117637	5.9	5.8	0.714
Algeria	2009	Developing	74.4	123	20	0.5	352.0636419	94	107	52.8	23	94	5.36	95	0.1	3868.83123	3546576	6	5.9	0.705



Regressão Múltipla
regressao_09.py



Regressão Múltipla - Gradiente Descendente
regressao_10.py

```
regressao_10.py
```

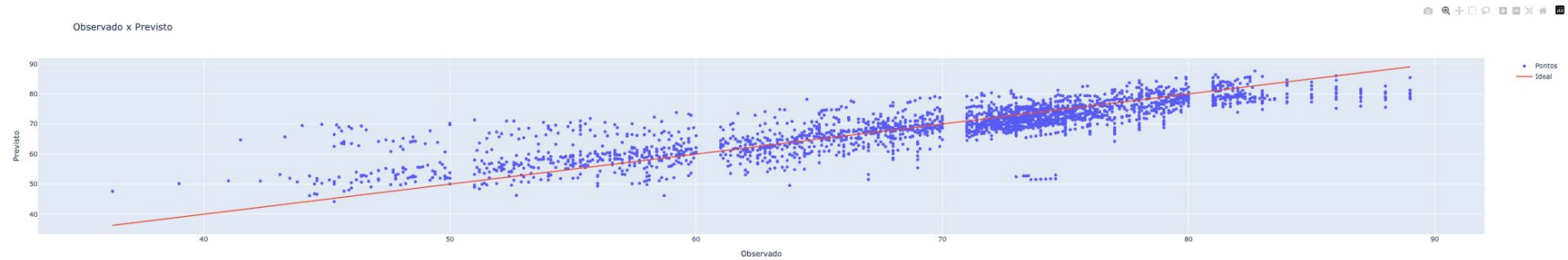
Regressão Múltipla - Django

Regressão Linear Múltipla

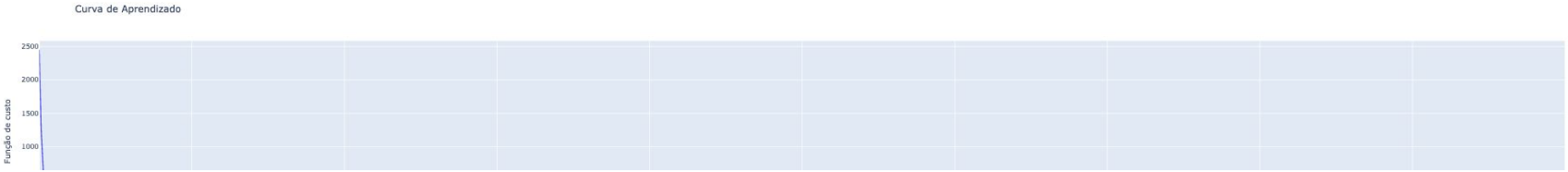
Indicadores

Intercepto	69.341845
Adult Mortality	-3.970533
Alcohol	-0.187708
Thinness	-0.946408
Schooling	4.911772
MSE	25.0033
R^2	0.7164

Observado x Previsto



Curva da Função de Custo



Um exemplo de previsão com dados do Kaggle

regressao_11

- Dados de imóveis da cidade de São Paulo (acessado em 01/08/2026)
  - https://www.kaggle.com/datasets/renatosn/sao-paulo-housing-prices/data

```
Caminho e nome do arquivo
    /Users/ronaldomartinsdacosta/am-env/python_django/am/regressao_11.csv
```

Um exemplo de previsão com dados do Kaggle

regressao_11

/Users/ronaldomartinsdacosta/am-env/python_django/am/templates/regressao_11.html

```
<!DOCTYPE html>
<html lang="pt-br">
<head>
    <meta charset="utf-8">
    <title>Regressão Linear - Imóveis - Regressão 11</title>
      ...
      ...
      ...
```

Um exemplo de previsão com dados do Kaggle

regressao_11

urls.py

```
/Users/ronaldomartinsdacosta/am-env/python_django/am/urls.py

from django.contrib import admin
from django.urls import path
from . import views

app_name = 'app_am'

urlpatterns = [
    path('', views.am_exemplo_01, name='am_exemplo_01'),
    path('am_exemplo_01', views.am_exemplo_01, name='am_exemplo_01'),
    path('regressao_01/', views.regressao_01, name='regressao_01'),
    path('regressao_05/', views.regressao_05, name='regressao_05'),
    path('regressao_10/', views.regressao_10, name='regressao_10'),
    path('regressao_11/', views.regressao_11, name='regressao_11'),
]
```

Um exemplo de previsão com dados do Kaggle

regressao_11

/Users/ronaldomartinsdacosta/am-env/python_django/am/views.py

```
def regressao_11(request):

      ...
      ...
      ...
```

Entendendo o exemplo construído

regressao_11

```
class sklearn.compose.ColumnTransformer(
    transformers,
    *,
    remainder='drop',
    sparse_threshold=0.3,
    n_jobs=None,
    transformer_weights=None,
    verbose=False,
    verbose_feature_names_out=True
)
```

Aplica transformações a colunas de um array ou de um DataFrame do pandas

Este estimador permite que diferentes colunas, ou subconjuntos de colunas da entrada, sejam transformados separadamente. Em seguida, as variáveis (features) geradas por cada transformador são concatenadas para formar um único espaço de atributos (feature space)

Isso é especialmente útil para dados heterogêneos ou organizados em colunas, pois possibilita combinar diferentes técnicas de extração de características ou transformações em um único transformador

Entendendo o exemplo construído


```
class sklearn.compose.ColumnTransformer(
    transformers,
    *,
    remainder='drop',
    sparse_threshold=0.3,
    n_jobs=None,
    transformer_weights=None,
    verbose=False,
    verbose_feature_names_out=True
)
```

remainder='drop'

É o comportamento padrão.
Todas as colunas que não forem listadas serão descartadas.

Entendendo o exemplo construído

regressao_11

```
class sklearn.compose.ColumnTransformer(
    transformers,
    *,
    remainder='drop',
    sparse_threshold=0.3,
    n_jobs=None,
    transformer_weights=None,
    verbose=False,
    verbose_feature_names_out=True
)
```

sparse_threshold=0.3,
Decide se a saída será
matriz esparsa (Sparse Matrix)
matriz densa (Dense Matrix)

Se menos de 30% dos elementos forem diferentes de zero →
matriz esparça

Entendendo o exemplo construído

regressao_11

```
class sklearn.compose.ColumnTransformer(
    transformers,
    *,
    remainder='drop',
    sparse_threshold=0.3,
    n_jobs=None,
    transformer_weights=None,
    verbose=False,
    verbose_feature_names_out=True
)
```

n_jobs=None

Controla o paralelismo, Usa todos os processadores disponíveis.

Entendendo o exemplo construído

regressao_11

```
class sklearn.compose.ColumnTransformer(
    transformers,
    *,
    remainder='drop',
    sparse_threshold=0.3,
    n_jobs=None,
    transformer_weights=None,
    verbose=False,
    verbose_feature_names_out=True
)
```

transformer_weights=None

Permite atribuir um peso diferente para a saída de cada transformador
Ou seja, cada saída é multiplicada pelo peso correspondente antes de ser concatenada

Entendendo o exemplo construído

regressao_11

```
class sklearn.compose.ColumnTransformer(
    transformers,
    *,
    remainder='drop',
    sparse_threshold=0.3,
    n_jobs=None,
    transformer_weights=None,
    verbose=False,
    verbose_feature_names_out=True
)
```

verbose=False

Mensagens em tela durante o processamento

Entendendo o exemplo construído

regressao_11

```
class sklearn.compose.ColumnTransformer(
    transformers,
    *,
    remainder='drop',
    sparse_threshold=0.3,
    n_jobs=None,
    transformer_weights=None,
    verbose=False,
    verbose_feature_names_out=True
)
```

verbose_feature_names_out=True

Este parâmetro controla como os nomes das colunas transformadas serão apresentados
O nome do transformador é usado como prefixo
transformers=[("cat", OneHotEncoder(), ["Sexo"])]
ct.get_feature_names_out()
Resultado:
cat__Sexo_F
cat__Sexo_M

Entendendo o exemplo construído

regressao_11

```
pre_processador = ColumnTransformer(
    transformers=[
        (
            "tipo",
            OneHotEncoder(
                categories=[[
                    "Apartamento",
                    "Casa",
                    "Casa em condomínio",
                    "Studio e kitnet"
                ]],
                drop=["Casa"],
                handle_unknown="ignore"
            ),
            ["type"]
        )
    ],
    remainder="passthrough"
)
```

Transforma os dados para serem processados, especialmente dados alfanuméricos em numéricos.
Por exemplo:

type	area	quartos
Casa	120	3
Apartamento	90	2
Studio e kitnet	35	1
Casa em condomínio	210	4

Seria transformado para:

Apartamento	Casa em condomínio	Studio	area	quartos
0	0	0	120	3
1	0	0	90	2
0	0	1	35	1
0	1	0	210	4

Entendendo o exemplo construído

regressao_11

```
modelo = Pipeline(
     [
          ("pre", pre_processador),
          ("modelo", LinearRegression())
     ]
 )
```

class sklearn.pipeline.Pipeline(steps, *, transform_input=None, memory=None, verbose=False)

O Pipeline aplica, de forma sequencial, uma lista de transformadores para realizar o pré-processamento dos dados e, se desejado, finalizar a sequência com um modelo preditivo para tarefas de aprendizado de máquina

```
modelo.fit(X, y)
```

Instrução que dispara o "treinamento" do modelo

fit(X, y=None, **fit_params)[source]

No método fit() o Scikit-Learn "aprende" os parâmetros necessários a partir dos dados
No caso de um Pipeline, o fit() não é executado apenas no LinearRegression. Ele percorre todas as etapas na ordem definida no pipeline
O objetivo do fit() é encontrar uma relação matemática entre X e y
Depois de executado o objeto modelo vai conter:

- médias do scaler
- categorias do encoder
- regras de transformação

Entendendo o exemplo construído

regressao_11

```
pred = modelo.predict(X)
```

Faz uma previsão para cada amostra, geralmente recebendo apenas X como entrada

Sempre será executado depois que um fit for executado

Qual seria o resultado?

```
pred = modelo.predict(x)
print(pred)
```

sendo X:

```
area,bedrooms,garage,type
150, 3, 2, Apartamento
```

Entendendo o exemplo construído

regressao_11

```
pred = modelo.predict(X)
```

Faz uma previsão para cada amostra, geralmente recebendo apenas X como entrada

Sempre será executado depois que um fit for executado

Qual seria o resultado?

```
pred = modelo.predict(x)
print(pred)
```

sendo X:

```
area,bedrooms,garage,type
120, 3, 2, Apartamento
```

6454.85

regressao_11

Previsão do Valor de Locação de Imóvel na Cidade de São Paulo

[Dados obtidos no kaggle - Acessado em 01/08/2026]

Área (m²)
120,0

Quantidade de quartos
3

Quantidade de vagas
2

Tipo do imóvel
Apartamento

Calcular previsão

Valor previsto

R$ 6454.85

Cinco imóveis com valores mais próximos

Endereço	Bairro	Área	Quartos	Garagem	Tipo	Valor
Rua Japuruchitá	Alto da Mooca	250	3	2	Casa	R$ 6455
Rua Said Aiach	Paraíso	56	1	1	Apartamento	R$ 6458
Rua Visconde de Aguiar Toledo	Vila Congonhas	300	3	2	Casa	R$ 6448
Rua Teodoro Sampaio	Pinheiros	78	2	1	Apartamento	R$ 6462
Rua Alves Guimarães	Pinheiros	75	1	1	Apartamento	R$ 6447

Modelo de Regressão Linear

Intercepto: -1055.0117

R^2: 0.5663

Variável	Coeficiente
tipo__type_Apartamento	2701.9796
tipo__type_Casa em condomínio	1285.2837
tipo__type_Studio e kitnet	2652.063
remainder__area	27.3397
remainder__bedrooms	7.0534
remainder__garage	752.9802

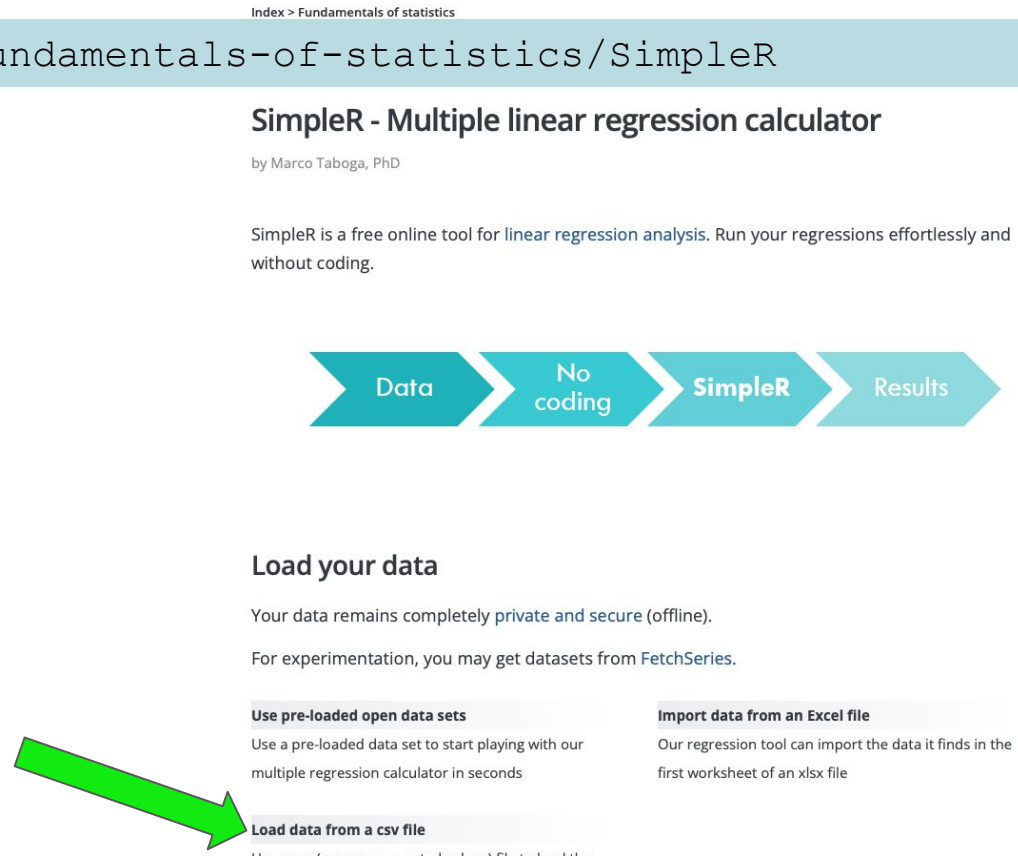


Verificando nossa Regressão

StatLect

regressao_11

https://www.statlect.com/fundamentals-of-statistics/SimpleR



Verificando nossa Regressão

Your data

The following checks were performed:

- **Header check**: there seems to be a proper header. Use Header +/- in the toolbar to modify.
- **Row check**: passed. All the rows have the same number of columns/fields.

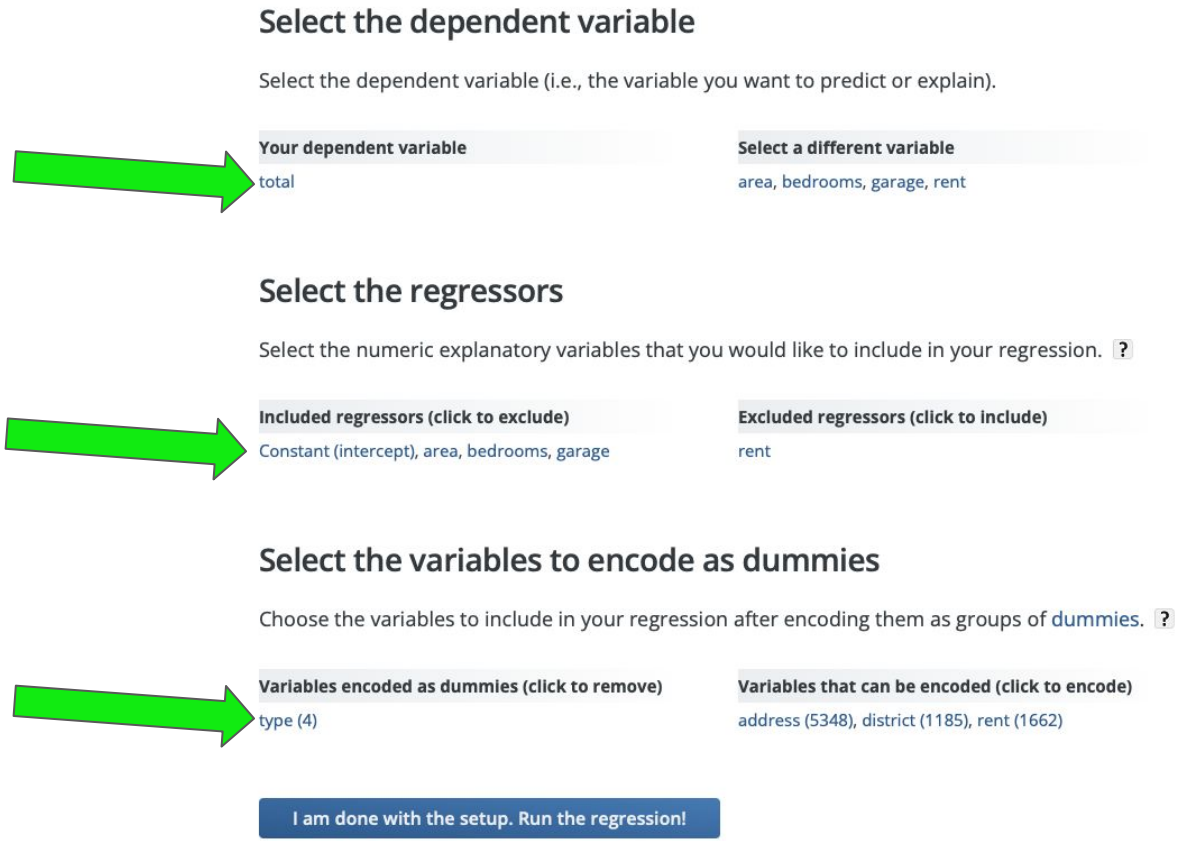
11657 observations | 8 variables | Header + / - | Analyze variables

▼ address	▼ district	▼ area	▼ bedrooms	▼ garage	▼ type	▼ rent	▼ total
Rua Herval	Belenzinho	21	1	0	Studio e kitnet	2400	2939
Avenida São Miguel	Vila Marieta	15	1	1	Studio e kitnet	1030	1345
Rua Oscar Freire	Pinheiros	18	1	0	Apartamento	4000	4661
Rua Júlio Sayago	Vila Ré	56	2	2	Casa em condomínio	1750	1954
Rua Barata Ribeiro	Bela Vista	19	1	0	Studio e kitnet	4000	4654
Rua Domingos Paiva	Brás	50	2	1	Apartamento	3800	4587
Rua Guararapes	Brooklin Paulista	72	2	1	Apartamento	3500	5187

Verificando nossa Regressão

StatLect
regressao_11

UFG INF INSTITUTO DE INFORMÁTICA



Verificando nossa Regressão

StatLect
regressao_11

UFG INF INSTITUTO DE INFORMÁTICA

Results

Here are the results from your regression.

Multicollinearity

Warning: the following variables were dropped to avoid perfect or almost perfect multicollinearity:

1. Dummy (type = Casa)

Sample size and degrees of freedom

Here is the calculation of the degrees of freedom (sample size - number of parameters).

Number of observations	Number of regressors	Degrees of freedom
11657	7	11650

Verificando nossa Regressão

StatLect
regressao_11

UFG INF INSTITUTO DE INFORMÁTICA

Coefficient estimates and their significance

In the next table, you can find the ordinary least squares (OLS) coefficient estimates, their standard errors and other statistics. ?

Variable name	Coefficient estimate	Standard error	t statistic	Two-sided p-value	One-sided p-value
Intercept	-1055	71.09	-14.84	0.000	0.000
area	27.34	0.4946	55.28	0.000	0.000
bedrooms	7.053	34.62	0.2037	0.8386	0.4193
garage	753.0	27.67	27.21	0.000	0.000
Dummy (type = Studio e kitnet)	2652	80.50	32.95	0.000	0.000
Dummy (type = Apartamento)	2702	53.24	50.75	0.000	0.000
Dummy (type = Casa em condomínio)	1285	148.3	8.668	4.975e-18	2.488e-18

Goodness of fit

And here are some statistics about the fit of the regression model. ?

R squared	Adjusted R squared
0.5663	0.5661

Universidade Federal de Goiás
Instituto de Informática
Prof. Dr. Ronaldo Martins da Costa

UFG
INF
INSTITUTO DE INFORMÁTICA

